

Legyen Ön Is Milliomos!

Dokumentáció

Kerekes László

USJ11R

Felhasználói dokumentáció: Legyen Ön Is Milliomos

Játék leírás:

Mint ahogy a tévéműsorban is a játékban is 15 kérdést kell teljesíteni a főnyeremény eléréséhez. Egy kérdéshez 4 válasz tartozik. A kérdések mind feleletválasztásosak, így, ha a játékos nem tudja rá a választ akár tippelhet is. Az első kérdést teljesítve 10000 Ft-tal lesz gazdagabb a felhasználó, majd ezek a nyeremények egyre nőnek és a végső nyereményig, ami 50 millió Ft. Ezt a 15 kérdés helyes megválaszolása után kaphatja meg a játékos. A játékban a kérdések egymást követik (helyes válaszok esetén), így nincs lehetőség se előre se hátrafelé ugrálni a kérdések között. A játékos használhat segítségeket 2 félét felezőt vagy a közönség segítségét.

A játéknak kétféleképpen lehet vége:

- ha a játékos egy kérdésre helytelen választ ad.
- ha teljesíti a 15 kérdést egyhuzamban ez esetben megnyeri a főnyereményt.

Felhasználói lehetőségek:

A megnyitás után 3 lehetőség van:

Játék: Ezt a lehetőséget választva egy kérdés elé állítja a program a játékost, mégpedig, hogy milyen nehézségű kérdésekkel szeretne találkozni a játék során. Létezik kezdő, haladó és profi szint is. Ezután egyből elindul a játék. Ezután egy enter nyomásával visszatérhetünk a menübe.

Dicsőség lista: Ezt a lehetőséget választva, az eddigi legjobb eredményeket elért emberek neveit, általuk nyert összeget és a játékban eltöltött időt fogjuk látni a megnyert összeg szerint csökkenő sorrendben. A listára bármilyen szintű játékot játszó játékos felkerülhet és egyenlő elbírálásban részesül mind a három szint. Ezután egy enter nyomásával visszatérhetünk a menübe.

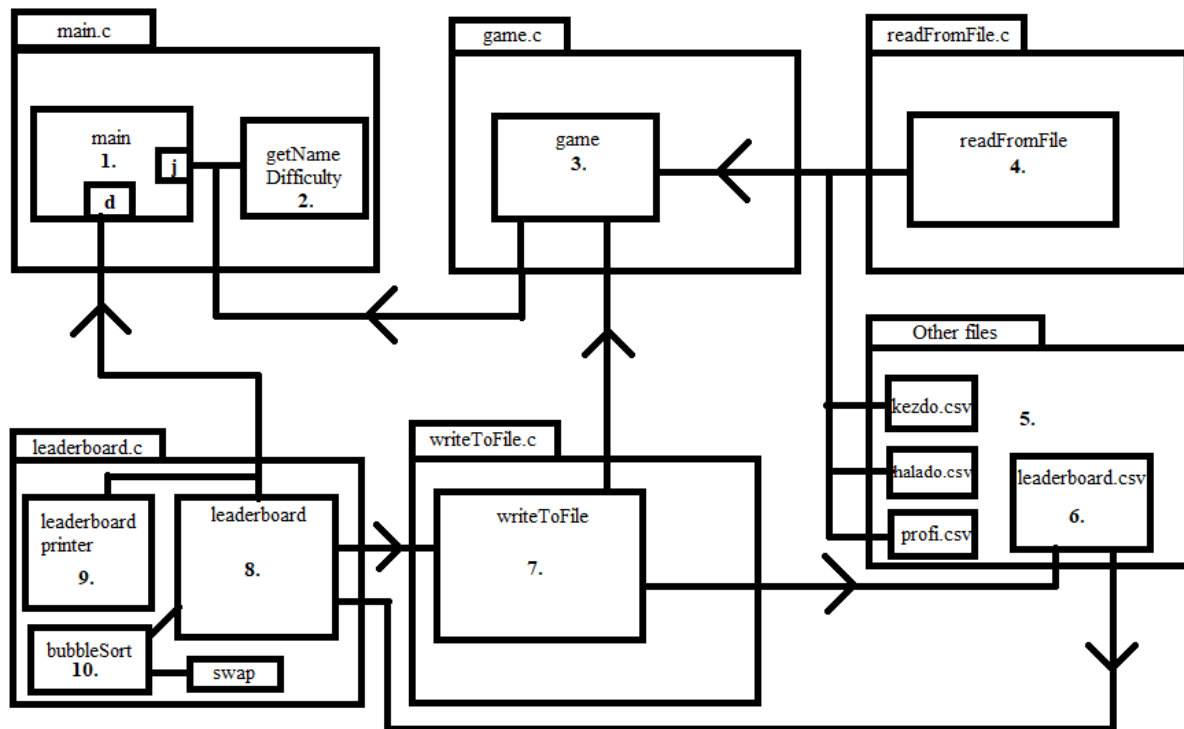
Kilépés: Ez a lehetőség kilép a programból.

Játék vezérlés:

A kérdésre a 4 válasz mindegyikének van egy betű jele, amit beírva tudunk válaszolni a kérdésre. A két segítség eléréséhez, követni kell a program utasításait, és i-t vagy n-t kell nyomni a megfelelő helyen. A segítség elhasználása után a program már nem fog rákérdezni, hogy kérjük-e őket. Játék előtt meg kell adni A játékos nevét és az elért eredmény alapján akár a dicsőség listára is felkerülhet a név, nyeremény, és a játékban töltött idő.

Programozói dokumentáció: Legyen Ön Is Milliomos

A Program vázlata:



Tartalomjegyzék

1. [main](#)
2. [getnameDifficulty](#)
3. [game](#)
4. [readFromFile](#)
5. [Használt adatbázisok](#)
6. [Dicsőségtábla](#)
7. [writeToFile](#)
8. [leaderboard](#)
9. [leaderboardprinter](#)
10. [bubbleSort](#)
11. [Technikai jellemzők](#)

A függvények darabonkénti bemutatása:

1. **main:**

A függvény menüként funkcionál az elején kiírja a főcímet, majd a lehetőségeket mellettük. Ezután scanf-el beolvasunk cMenuOption nevű karakter változóba, amit egy switchben használunk.

Az első lehetőség a 'j' mint játék, itt meghívjuk a [getNameDifficulty](#) függvényt, amivel a felhasználótól kapunk egy nevet string formában, valamint a függvény visszatérési értékében egy betűt, ami a nehézség választás lesz. Ezekkel a kapott adatokkal meghívjuk a game függvényt, ami elkezd a játékot, majd, ha annak vége visszatér ide, ahol az enter megnyomásával továbbmehetünk és visszajutunk a menübe.

A második lehetőség a 'd' választásával nyílik meg ez a dicsőségtábla, amiben először a [leaderboard](#) nevű függvényt hívjuk, aminek adunk egy [dynamicLeaderboard](#) típusú változót ezt a későbbiekben jobban kifejtem, a lényeg, hogy a meglévő adatokat kiolvassa a leaderboard.csv fájlból és azokat visszaadja. Ezt átadjuk a [leaderboardprinter](#) függvénynek, ami kiírja nekünk őket, innen is enter lenyomásával juthatunk vissza a menübe.

A harmadik lehetőség a 'k' betű választásával érhető el ez az opció a kilépés, amivel a programból kilépünk és láthatjuk a debugmalloc ellenőrzés eredményét.

Akármilyen más betű esetén újra a menüt látjuk és újra választhatunk.

2. **getNameDifficulty:**

Ez a függvény a menü játékrészének segédfüggvénye, bemenetként egy karaktertömböt kap, majd kér egy nevet a felhasználótól, az ékezetes betűket is tudja kezelni a program. A név beolvasásához egy ideiglenes karakter tömböt használ amiből utána ezt bemásolja a kapott tömbbe és végén lévő entert '\0'-ra cseréli.

Utána a nehézséget kérdezzük meg, ami, 'k', 'h' és 'p' lehet, ha ezek közül egyik sem akkor visszakérülünk a kérdés elé és újra megkérdezzük amíg nem lesz ezek közül valamelyik, ha megvan visszatérünk vele.

3. **game:**

Ez a függvény maga a játék, két bementi paramétere van az sName és a cDifficulty. A kapott cDifficulty alapján meghívja a [readFromFile](#) függvényt a megfelelő fájlnevével, innen egy [dynamicArray](#) típusú struktúrában kapja meg a kérdéseket dinamikusan lefoglalva.

Létrehozunk sok segéd változót:

aiRewards: a kérdés számához tartozó nyereményeket tartalmazó tömb.

iQuestionindex: egy véletlenszerű szám az, amit a kérdések száma alapján generálunk.

iRewardIndex: a továbbiakban ez fog a kérdés számlálóként működni, ez megy az

aiRewards tömbön belül is.

([QuestionDataModel](#)) model: egy kérdést tároló struktúra, ami mindig az aktuális kérdéssel van felülírva.

cHalfHelper: karakter, amiben az i-t vagy n-t tároljuk, a felező segítséghez használjuk.

cAudienceHelper: karakter, amiben az i-t vagy n-t tároljuk, a közönség segítséghez használjuk.

(bool) bDidUseHalf, (bool) bDidUseAudience: ezeket arra használjuk, hogy csak egyszer lehessen használni a segítségeket.

Ezután elindítunk egy stoppert, ami méri a játék végéig eltelt időt.

Innen indul maga játék egy do while funkcióval, ami addig megy ameddig a felhasználó nem ad rossz választ vagy nem jut el a 15. utolsó kérdéshez.

A kérdést kapott [dynamicArray](#) struct megfelelő ([QuestioDataModel](#)) questions tömbből kapjuk.

Az indexet az elején növeljük minden lefutásnál, majd kiirunk egy kérdést, amit egy random szám alapján kapunk a következő kérdés mindig egyel nagyobb indexen lévő, hogy elkerüljük az ismétlődést, ha esetleg túlfutna újra modulózzuk a kérdések számával.

Itt megkérdezzük a felhasználót akar a segítséget igénybe venni, ez mindkét segítséggel lefut ameddig azokat el nem használták, ha nem akkor válaszolnia kell, a választ a betűjellel kell megadni, kicsi és nagy betű is elfogadott.

A segítségek:

A kérdés, hogy akarunk-e segítséget használni mindkét esetben az elhasználásig minden kérdésben megjelenik, a segítség csak akkor használódik fel, ha i-t kap különben nem, más karakter nem okoz gondot viszont nemként van elbírálva.

Az első felező segítség, ami kivesz kettő rossz megoldást és utána újra kiírja a lehetőségeket, mindegyik válaszlehetőséghez tartozik egy segéd bool hogy azt a végén kiírjuk-e. Először is a jó választ igazra állítjuk, majd egy a jó választól eltérő random betűnek is igazra állítjuk ha ez megvan újra kiírjuk a kérdést a megmaradó válaszlehetőségekkel. A két segítség egyszerre nem használhatóságát egy boolal oldottam meg.

A második a közönség segítése, ami egy százalékot ad minden válasza, létrehozunk négy segéd (int) számlálót minden válaszlehetőséghez. Ezután van egy ciklus, ami 100-szor fut le, hogy ne kelljen osztogatni az eredményeket, ezen belül számot randomizálunk 1 és 5 között, az első négy a megfelelő számlálóhoz hozzáad egyet az ötödik pedig mindig a jó megoldáshoz ad egyet. Ezután kiírjuk a válaszlehetőségeket mellettük a százalékkal, hogy mennyi az esély, hogy az a jó megoldás, nyilván a jó megoldás lesz a legmagasabb.

A játék végén az időzítő megáll és az időt elmentjük, majd átadjuk a névvel és nyert összeggel együtt a [writetoflie](#) nevű függvénynek, valamint felszabadítjuk a kapott pointerben lefoglalt memóriát. Informáljuk a felhasználót a nyert összegről, ha esetleg megnyerte a főnyereményt akkor ehhez gratulálunk.

4. [readFromFile](#):

Ez a függvény a kérdések fájlokból való kiolvasásával foglalkozik, egy fájlnevet kap stringként, és egy dynamicArray típusú pointert ad vissza. A dynamicArray kérdéseket tárol, ebben a struktúrában egy QuestionDataModel típusú tömböt és a kérdések mennyiségét intigerként tárolja.

A QuestionDataModel pedig egy kérdés minden információját tárolja ezek a következők: Maga a kérdés stringként, a kérdés típusa, a válaszok és a helyes válasz betűjele. A program először leszámolja hány sor van a fájlban, a jelenlegi adatbázisok nem haladják viszont van benne egy memória használat limit (2500 sor), lefoglal ennyi struktúrának memóriát, utána újra megnyitja a fájlt és beolvassa a sorokat stringként majd ezt a megadott formátum alapján beolvassa a struktúrába soronként. Az itt lefoglalt memóriát később fel kell szabadítani, amit a program a game függvény végén meg is tesz.

5. Használt adatbázis fájlok:

A három nehézségi szinthez három külön fájl tartozik ezek, ha tartjuk a formátumot kicserélhetőek viszont akkor a game függvényben át kell írni a fájl nevet, amit az adott nehézséghez akarunk használni. A formátum a csv így, ha Excelből MS-DOS konvertert használunk akkor az ékezeteket is tudja kezelni a program, az egyes adatokat ';' választja el, fontos megjegyezni hogy a sorok száma limitált (2500). Ezek a következő sorrendben vannak: A kérdés, egyesével a négy választási lehetőség, a jó válasz nagy betűvel, a válasz típusa.

6. Dicsőségtábla:

Ezekon kívül a leaderboard.csv fájlt használja a program, amibe az egyes játékosok eredményeit jegyzi le minden játék végén, a fájl tartalmazza a játékos nevét a nyereményének összegét, valamint a játék közben eltelt időt. Ezt a fájlt azért nem hozza létre a program maga, mert ékezetek kezelése miatt Excelből lett konvertálva és ilyet nem tud létrehozni. A fájl mérete limitálva van 50 adatra, de a levágás egy sorba rendezés után történik így a kevésbé releváns adatok tűnnek el.

7. writeToFile:

Ez a függvény a game függvényből van hívva a feladata, hogy a lejátszott játék statisztikáját bevigye a leaderborad.csv fájlba, mielőtt ezt megtehetné a már benne lévőket megkapja sorba rendezve a [leaderboard](#) függvénytől, csak a legjobb 49 eredmény marad bent, ezeket újra beírja és a végére odateszi a legújabbat. Felszabadítja az összes kapott adatot figyelve azokra is, amik nem kerülnek a fájlba vissza írásra.

8. leaderboard:

Ez a függvény a dicsőségtábla adatait hivatott a fájlból kiolvasni, kezeli a fájl meg nem létét és kilép, ha nem találja. Először is leszámolja hány sor adat van aztán beolvassa a sorokat [dynamicleaderboard](#) struktúrába, előtte foglal is nekik memóriát, ezt a pointert később visszaadja, előtte még meghív egy [bubbleSort](#) nevű függvényt, ami sorba rendezi az adatokat.

A függvény használ adattároló stuktúrákat:

[dynamicLeaderboard](#): Gyakorlatilag a dinamikus tömb amiben [leaderboard](#) tömb van és az adatok száma integerként.

[leaderboard](#): egy játék adatait tartalmazza a játékos nevét stringben, a nyereményét, az időt amennyit játszott.

9. leaderboardprinter:

Ez függvény a kapott [dynamicLeaderboard](#) struktúrában lévő adatokat kiírja, ez a dicsőségtábla látható részét alkotja, a [mainból](#) van hívva és már sorba tett adatokat kap, a kapott struktúrán belül mindent felszabadít.

10. bubbleSort:

A függvény egy hatékony szortírozó algoritmus, aminek a cserélendő tagok számától függ a futási ideje, jelen esetben az adatok nyert összeg szerinti sorba rendezése a feladata, van egy segéd függvénye (swap), ami kicserél két adatot.

A Program technikai jellemzői

A futtatáshoz az NHFcodeblocks.exe használható,
ezen kívül a négy adatbázis: kezdó.csv, halado.csv, profi.csv, leaderboard.csv.

Az újra lefordításhoz kellő fájlok: game.c, leadeboard.c, main.c, readfromfile.c,
writetoflie.c, debugmalloc.h, game.h, leaderboard.h, readfromfile.h, writetofile.h,
NHFcodeblocks.cbp és a négy adatbázis: kezdó.csv, halado.csv, profi.csv, leaderboard.csv.

Maximális memória igény: Futás közben nagyjából 1,35Mb.

Maximális CPU igény: Az én processzoromon (Ryzen 5 4600H) elhanyagolható (0%) volt a
használat futás közben

A futási idő teljesen a felhasználótól függ, akkor lép ki, amikor akar, ha nem volt hiba a
futás közben.

Ez program a Codeblocks 20.03-as verziójában készült, ugyanazokkal a beállításokkal
mint a benti gépeken (Wall ,Werror), ez a dokumentum a kód 1.0-ás verziójához készült,
ami az első teljes verzió.